

컴퓨터 구조 강의 필기노트

교재 《혼자 공부하는 컴퓨터 구조 + 운영체제》(강민철)

범위 00:00:00 ~ 03:26:13

INTRO 0. 오리엔테이션

강의 및 교재 소개

- 강민철 저자의 《혼자 공부하는 컴퓨터 구조 + 운영체제》 기반 강의
- 학부 전공서적(패터슨 컴퓨터 구조, 공룡책 등)에서 개발자에게 꼭 필요한 핵심만 추출 → 교수·현업 개발자 감수 완료
- 이론·개념 위주 강의이므로 **교재 병행 학습 권장** (생소한 용어 복습용)
- 선행 지식 없이도 이해 가능하나, 변수·조건문·함수 등 기초 프로그래밍 문법을 알면 더 넓은 시야로 수강 가능

컴퓨터 구조를 알아야 하는 이유

1 문제 해결 능력 배양

- 컴퓨터를 '미지의 대상'이 아닌 '분석의 대상'으로 보기
- 코드는 맞는데 오류가 날 때, 내부 구조(메모리·CPU·버스)를 알아야 원인 분석 가능

2 성능·용량·비용을 고려하는 개발자

- 프로그램의 메모리·CPU 자원 사용을 이해해야 클라우드 비용 최적화·고성능 코드 작성 가능

CH.01 컴퓨터 구조 시작하기

1.1 컴퓨터가 이해하는 두 가지 정보

구분	설명
데이터 (Data)	컴퓨터가 이해하는 정적인 정보 (숫자, 문자, 이미지, 동영상, 파일 등)
명령어 (Instruction)	컴퓨터를 실질적으로 작동시키는 정보. 데이터는 명령어 수행의 재료

☞예시 — "1과 2를 더하라" → **1**, **2** 는 데이터 / '더하라'는 명령어

1.2 컴퓨터의 4가지 핵심 부품

컴퓨터는 **CPU · 메모리(주기억장치) · 보조기억장치 · 입출력장치**로 구성되며, 메인보드의 **시스템 버스**로 연결된다.

① CPU (Central Processing Unit) — 컴퓨터의 뇌

- ALU (산술논리연산장치)**: 계산 담당
- 제어장치 (Control Unit)**: 제어 신호 전송 + 명령어 해석
- 레지스터 (Register)**: CPU 내부 임시 저장장치, 데이터·명령어를 빠르게 저장

② 메모리 (주기억장치, Primary Memory)

- 본 강의에서 '메모리' = 주로 **RAM**을 지칭 (ROM 포함되나 RAM 위주 설명)
- 특징: 현재 실행 중인 프로그램의 명령어·데이터만 저장
- 단점: **휘발성(Volatile)** — 전원이 꺼지면 정보 소실
- 주소(Address)**: 메모리 내 저장 위치를 찾기 위한 번지수

③ 보조기억장치 (Secondary Storage)

- 비휘발성(Non-volatile) — 전원이 꺼져도 내용 유지 (예: SSD, HDD, USB)
- 실행되지 않은 프로그램 및 장기 보관 데이터 저장

④ 입출력장치 (Input/Output Devices)

- 컴퓨터 외부에서 인간/외부 환경과 정보를 주고받는 장치 (키보드, 마우스, 모니터, 프린터, 스피커 등)

⑤ 메인보드와 시스템 버스 (System Bus)

- 메인보드: 핵심 부품이 연결되는 판
- 시스템 버스 (3종류)
 - 주소 버스 (Address Bus) — 주소 정보 전달
 - 데이터 버스 (Data Bus) — 데이터·명령어 전달
 - 제어 버스 (Control Bus) — 제어 신호 전달

CH.02 데이터 (0과 1의 표현)

2.1 0과 1로 숫자를 표현하는 방법

1 정보 단위

- 비트(bit): 0 또는 1, 가장 작은 정보 단위 → n비트로 2^n 가지 정보 표현
- 바이트(Byte): 8비트 = 1 Byte
- 단위 확장: **1 KB = 1024 Bytes = 2^{10} Bytes** (일상 단위 1000과 차이 있음 → KiB/MiB로 구분)
- 확장 순서: KB → MB → GB → TB → PB

2 진법

- 2진법(Binary): 0과 1만으로 수 표현
- 16진법(Hexadecimal): 0~9, A~F(10~15) 사용
 - 컴퓨터에서 16진법을 쓰는 이유: 2진수의 길고 복잡한 표기를 간결화 (16진수 1자리 = 2진수 4비트, 1:1 대응)

3 음수 표현법 — 2의 보수법

- 컴퓨터는 부호(+/-)를 직접 저장 불가 → **2의 보수(2's complement)** 사용
- 만드는 법: 모든 비트 반전($0 \leftrightarrow 1$) 후 +1
- 플래그(Flag): 플래그 레지스터가 양수/음수 여부를 기록해 보정

2.2 0과 1로 문자를 표현하는 방법

1 용어 정의

- 문자 집합(Character Set): 컴퓨터가 이해하는 문자들의 모음
- 문자 인코딩(Encoding): 문자 → 0과 1의 이진 코드 변환
- 문자 디코딩(Decoding): 이진 코드 → 사람이 읽는 문자로 변환

2 주요 문자 인코딩 방식

① 아스키(ASCII) 코드

- 초창기 영문자 표준 인코딩
- 7비트 사용 → 총 $128(2^7)$ 개 문자 표현 (영문자·숫자·특수문자·제어문자)
- 한계: 한국어·한자 등 비영어권 언어·다양한 기호 표현 불가

② EUC-KR

- 한국어 표현용 **2바이트(16비트)** 기반 완성형 인코딩

- 초성·중성·종성 조합으로 완성된 글자마다 고유 코드 부여
- 한계: 표현 불가능한 한글 존재 (예: 뽕, 탕 등)

③ 유니코드(Unicode)와 UTF-8

- **유니코드**: 전 세계 모든 언어·기호에 고유 코드값(코드 포인트) 부여
- **UTF-8**: 유니코드를 0과 1로 변환하는 **가변 길이** 인코딩 방식
 - 문자에 따라 1~4바이트 가변 할당 → 공간 효율 ↑ (영문/아스키 영역 1바이트, 한글 주로 3바이트)
 - 현재 웹·시스템 표준 인코딩으로 가장 널리 사용

CH.03 명령어 (Instruction)

3.1 소스 코드와 명령어

1 고급 언어 vs 저급 언어

구분	설명
고급 언어 High-Level	사람이 이해·작성하기 쉬운 언어 (C, C++, Java, Python 등)
저급 언어 Low-Level	컴퓨터가 직접 이해·실행하는 언어

- **기계어(Machine Code)**: 0과 1로만 이루어진 명령어
- **어셈블리어(Assembly Language)**: 기계어를 사람이 읽기 쉬운 문자로 1:1 대응시킨 언어

2 고급 언어 → 저급 언어 변환 방식

① 컴파일 언어 (Compiled Language)

- 소스 코드 전체를 **컴파일러**가 한 번에 저급 언어(목적 코드)로 변환 후 실행
- 대표 언어: C, C++
- 특징: 실행 속도 빠름 / 수정 시 전체 재컴파일 필요

② 인터프리터 언어 (Interpreted Language)

- 소스 코드를 **인터프리터**가 한 줄씩 해석하며 실행
- 대표 언어: Python
- 특징: 실행 속도는 상대적으로 느림 / 오류 발생 지점 파악 용이

3.2 명령어의 구조

1 연산 코드와 오퍼랜드

하나의 명령어 = **연산 코드(Opcode)** + **오퍼랜드(Operand)**

- **연산 코드**: 명령어가 수행할 연산 (예: **LOAD** 데이터 불러오기, **STORE** 저장하기, **ADD** 더하기, **JUMP** 지정 주소로 이동)
- **오퍼랜드**: 연산에 사용할 데이터 또는 그 데이터의 저장 위치(메모리 주소/레지스터명) — '주소 필드'라고도 함

2 주소 지정 방식 (Addressing Modes)

오퍼랜드에 데이터 값 자체를 담으면 크기 제약 발생 → 실제로는 **데이터가 위치한 주소**를 담음 → 이 주소를 찾는 방식

📌 **유효 주소(Effective Address)** — 연산에 사용될 데이터가 실제 저장된 위치·주소

방식	설명	특징
① 즉시 주소 지정 Immediate	연산할 데이터 자체를 오퍼랜드 필드에 직접 명시	속도 매우 빠름 / 데이터 크기가 오퍼랜드 비트 수로 제한
② 직접 주소 지정 Direct	오퍼랜드 필드에 유효 주소(메모리 주소)를 직접 지정	표현 가능한 메모리 주소 범위에 한계
③ 간접 주소 지정 Indirect	오퍼랜드 필드에 '유효 주소가 저장된 메모리 주소'를 지정	주소 범위 확장 / 메모리 2회 접근 → 속도 느림
④ 레지스터 주소 지정 Register	오퍼랜드 필드에 데이터가 저장된 레지스터 이름을 지정	CPU 내부 접근 → 직접 주소 지정보다 훨씬 빠름
⑤ 레지스터 간접 주소 지정 Register Indirect	오퍼랜드 필드에 '유효 주소가 저장된 레지스터'를 지정	메모리 1회 접근 + 주소 범위 확장

CH.04 CPU의 작동 원리

4.1 ALU와 제어장치

① ALU (산술논리연산장치)

- 입력
 - 레지스터로부터 전달받은 피연산자(데이터)
 - 제어장치로부터 전달받은 제어 신호(연산 수행 명령)
- 출력
 - 연산 수행 결과 데이터 → 레지스터에 저장
 - 플래그(Flag) → 연산 결과의 부가 상태 정보

플래그 레지스터 (Flag Register) — ALU 연산 결과의 부가 정보 저장

플래그	의미
부호 플래그 Sign Flag	연산 결과가 음수인지 여부
제로 플래그 Zero Flag	연산 결과가 0인지 여부
오버플로우 플래그 Overflow Flag	결과가 표현 가능 범위를 초과했는지 여부
캐리 플래그 Carry Flag	올림수·내림수 발생 여부
인터럽트 플래그 Interrupt Flag	인터럽트 허용 여부

② 제어장치 (Control Unit)

CPU 내부·외부 부품에 제어 신호를 보내고 명령어를 해석하는 컨트롤러

- 받아들이는 정보
 - 클럭(Clock) 신호 — 컴퓨터의 박자에 맞춰 동작하기 위한 주기적 전기 신호
 - 명령어 레지스터(IR)의 명령어 — 해석 대상
 - 플래그 레지스터의 플래그 값 — 연산 결과 상태 참고
 - 제어 버스에서 도달한 제어 신호 — 외부 부품(입출력장치 등)에서 전달

- **내보내는 정보**

- CPU 내부: ALU(연산 종류 지시), 레지스터(데이터 이동 지시)
- CPU 외부: 메모리 읽기/쓰기 신호, 입출력장치 제어 신호

4.2 레지스터의 종류와 역할

레지스터	역할
프로그램 카운터 PC, Program Counter	메모리에서 읽어들일 다음 명령어의 주소 저장 (일부 아키텍처는 IP, Instruction Pointer)
명령어 레지스터 IR, Instruction Register	메모리에서 읽어온, 현재 해석·실행할 명령어를 임시 저장
메모리 주소 레지스터 MAR	CPU가 읽거나 쓸 메모리 주소를 주소 버스로 보내기 전 저장
메모리 버퍼 레지스터 MBR / MDR	메모리와 주고받을 데이터·명령어를 데이터 버스로 보내기 전/후 저장
범용 레지스터 General Purpose Register	데이터·주소를 자유롭게 저장하는 다목적 레지스터
플래그 레지스터 Flag Register	ALU 연산 결과 상태 / CPU 동작 상태 정보 저장

🔴 03:26:13 **지점 메모** — 레지스터의 기본 역할 설명을 마친 직후 구간. 다음 내용은 **스택 포인터·베이스 레지스터를 이용한 주소 지정 방식** 및 **명령어 사이클 / 인터럽트** 단원으로 이어짐.

✓ 한눈에 보기 (핵심 정리)

- 컴퓨터 = **데이터** + **명령어**로 작동
- 4대 부품: **CPU** / **메모리** / **보조기억장치** / **입출력장치** — 시스템 버스(주소·데이터·제어)로 연결
- 음수는 **2의 보수**, 문자는 **ASCII → EUC-KR → 유니코드(UTF-8)** 순으로 발전
- 명령어 = **연산 코드** + **오퍼랜드**, 오퍼랜드는 **주소 지정 방식**(즉시/직접/간접/레지스터/레지스터 간접)으로 데이터 위치를 찾음
- CPU 내부 = **ALU** + **제어장치** + **레지스터**(PC, IR, MAR, MBR, 범용, 플래그)